

```

# =====

# Sentence-Weighted Sentiment Analysis using FinBERT

# Author: Dev Kinger

# We shall do the following:

# - Split section text into sentences

# - Score each sentence using FinBERT

# - Aggregate sentiment using sentence-length weights

# =====


# -----

# Imports

# -----

import pandas as pd

import numpy as np

import re

from transformers import pipeline


# -----

# Configuration

# -----

INPUT_FILE = "input_data.xlsx"

OUTPUT_FILE = "Sentiment_SentenceWeighted.xlsx"


# -----

# Load FinBERT Model

# -----

print("Initialising FinBERT sentiment model...")


# Notes:

# - device = -1 → CPU execution (safe default)

```

```

# - top_k = None → return probabilities for all labels

classifier = pipeline(
    task="sentiment-analysis",
    model="ProsusAI/finbert",
    top_k=None,
    device=-1
)

# -----

# Load Input Data

# -----

print(f"Reading input file: {INPUT_FILE}")

df = pd.read_excel(INPUT_FILE)

required_columns = ["Section_name", "Section_text", "Year"]
missing_cols = [c for c in required_columns if c not in df.columns]
if missing_cols:
    raise ValueError(f"Missing required columns: {missing_cols}")

# Drop empty text rows early
df = df.dropna(subset=["Section_text"]).reset_index(drop=True)

# =====

# Sentence Processing Utilities

# =====

def split_sentences(text):
    """
    Splits raw section text into sentences.

    Also removes basic HTML artefacts if present.
    """

```

```
clean_text = re.sub(r"<[^>]+>", " ", str(text))
```

```
clean_text = " ".join(clean_text.split())
```

```
# Simple rule-based sentence splitting
```

```
sentences = re.split(r"(?<=[.!?])\s+", clean_text)
```

```
return [s for s in sentences if len(s.strip()) > 0]
```

```
def get_sentiment_probs(sentence):
```

```
    """
```

```
    Returns (positive, negative, neutral) probabilities  
    from FinBERT for a single sentence.
```

```
    """
```

```
    try:
```

```
        output = classifier(
```

```
            sentence,
```

```
            truncation=True,
```

```
            max_length=512
```

```
        )
```

```
# Convert list-of-dicts to label → score mapping
```

```
scores = {
```

```
    item["label"].lower(): item["score"]
```

```
    for item in output[0]
```

```
}
```

```
return (
```

```
    scores.get("positive", 0.0),
```

```
    scores.get("negative", 0.0),
```

```
    scores.get("neutral", 0.0)
```

```
)
```

```
except Exception as e:
```

```
    # Fail safely on rare model errors
```

```
    print(f"FinBERT error on sentence: {sentence[:40]}...")
```

```
    return 0.0, 0.0, 0.0
```

```
# =====
```

```
# Sentence-Level Aggregation Logic
```

```
# =====
```

```
def analyze_section_sentiment(section_text):
```

```
    """
```

```
    Computes sentence-weighted sentiment scores for one section.
```

```
    Weighting scheme:
```

```
        weight(sentence) = len(sentence) / len(section)
```

```
    """
```

```
    sentences = split_sentences(section_text)
```

```
    if not sentences:
```

```
        return pd.Series([np.nan, np.nan, np.nan, np.nan])
```

```
    char_lengths = [len(s) for s in sentences]
```

```
    total_chars = sum(char_lengths)
```

```
    if total_chars == 0:
```

```
        return pd.Series([0.0, 0.0, 0.0, 0.0])
```

```
    weighted_pos = 0.0
```

```
    weighted_neg = 0.0
```

```
weighted_neu = 0.0
```

```
for sent, n_chars in zip(sentences, char_lengths):
```

```
    pos, neg, neu = get_sentiment_probs(sent)
```

```
    weight = n_chars / total_chars
```

```
    weighted_pos += weight * pos
```

```
    weighted_neg += weight * neg
```

```
    weighted_neu += weight * neu
```

```
net_sentiment = weighted_pos - weighted_neg
```

```
return pd.Series([
```

```
    weighted_pos,
```

```
    weighted_neg,
```

```
    weighted_neu,
```

```
    net_sentiment
```

```
])
```

```
# =====
```

```
# Run Sentiment Analysis
```

```
# =====
```

```
print("Running sentence-weighted sentiment analysis...")
```

```
df[[
```

```
    "Score_Pos",
```

```
    "Score_Neg",
```

```
    "Score_Neu",
```

```
    "Net_Sentiment"
```

```
]] = df["Section_text"].apply(analyze_section_sentiment)
```

```
# -----
```

```
# Save Output
```

```
# -----
```

```
df.to_excel(OUTPUT_FILE, index=False)
```

```
print(f"Analysis completed → {OUTPUT_FILE}")
```